

LABORATORY MANAGEMENT SYSTEM

23CS54C - MODERN WEB TECHNOLOGIES LABORATORY

Submitted by

SUBASH MUTHU B	2312401
SUBIKSHAN M	2312501
ALWIN IMMANUEL J	2312404

In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



NATIONAL ENGINEERING COLLEGE
(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

APRIL - 2025

NATIONAL ENGINEERING COLLEGE
(An Autonomous Institution affiliated to Anna University, Chennai)
K.R.NAGAR, KOVILPATTI - 628503



BONAFIDE CERTIFICATE

This is to certify that this project report, “Laboratory Management System”, is the bonafide work of **SUBASH MUTHU B (2312401)**, **SUBIKSHAN M (2312501)**, **ALWIN IMMANUEL J (2312404)** who carried out the project work under my supervision.

Course Instructor/Guide

Submitted to the **23CS54C - MODERN WEB TECHNOLOGIES LABORATORY** Viva- Voce examination held at National Engineering College, K.R.Nagar, Kovilpatti on _

Internal Examiner

Co Examiner

TABLE OF CONTENTS

CH NO	TITLE	PAGE NO
1	INTRODUCTION	4
2	OBJECTIVES	5
3	DESCRIPTION	6
4	CLASS DIAGRAM	7
5	TABLE STRUCTURES	8
6	ER DIAGRAM	9
7	TECH STACK	10
8	MODULES	13
9	CONCLUSION	69

CHAPTER 1

INTRODUCTION

LabMS is a comprehensive web-based Lab Management and Maintenance System designed for academic and research institutions. The project aims to streamline the management of laboratory resources, including visitor registration, equipment booking, and maintenance scheduling. By integrating multiple modules into a unified platform, the system enhances operational efficiency, security, and transparency. The application supports role-based access for Admins, Hosts, and Lab Technicians, ensuring that each user interacts with the system according to their responsibilities. The system leverages modern web technologies, including a React-based frontend and a Spring Boot backend, to deliver a responsive and scalable solution. The project addresses common challenges in lab management such as manual record-keeping, scheduling conflicts, and lack of centralized reporting.

CHAPTER 2

OBJECTIVES

- **Role-Based Access Control:** Implement secure authentication and authorization for different user roles (Admin, Host, Lab Technician), ensuring data privacy and operational integrity.
- **Visitor Management:** Digitize visitor registration and approval workflows, allowing hosts to manage visitor entries and statuses efficiently.
- **Equipment Inventory & Booking:** Maintain a detailed inventory of laboratory equipment and facilitate booking requests, preventing scheduling conflicts and ensuring optimal resource utilization.
- **Maintenance Scheduling:** Enable professional scheduling of equipment maintenance, including routine, preventive, and emergency tasks, with status tracking and priority assignment.
- **Dashboard & Reporting:** Provide real-time dashboards and date-range reports for admins and technicians, offering insights into lab usage, maintenance activities, and visitor trends.
- **CRUD Operations:** Support create, read, update, and delete operations for users, visitors, equipment, bookings, and maintenance records.
- **User Experience:** Design a responsive and intuitive frontend interface, with confirmation dialogs, notifications, and error handling for a seamless user experience.
- **Scalability & Security:** Ensure the system is scalable for large institutions and secure against unauthorized access and data breaches.

CHAPTER 3

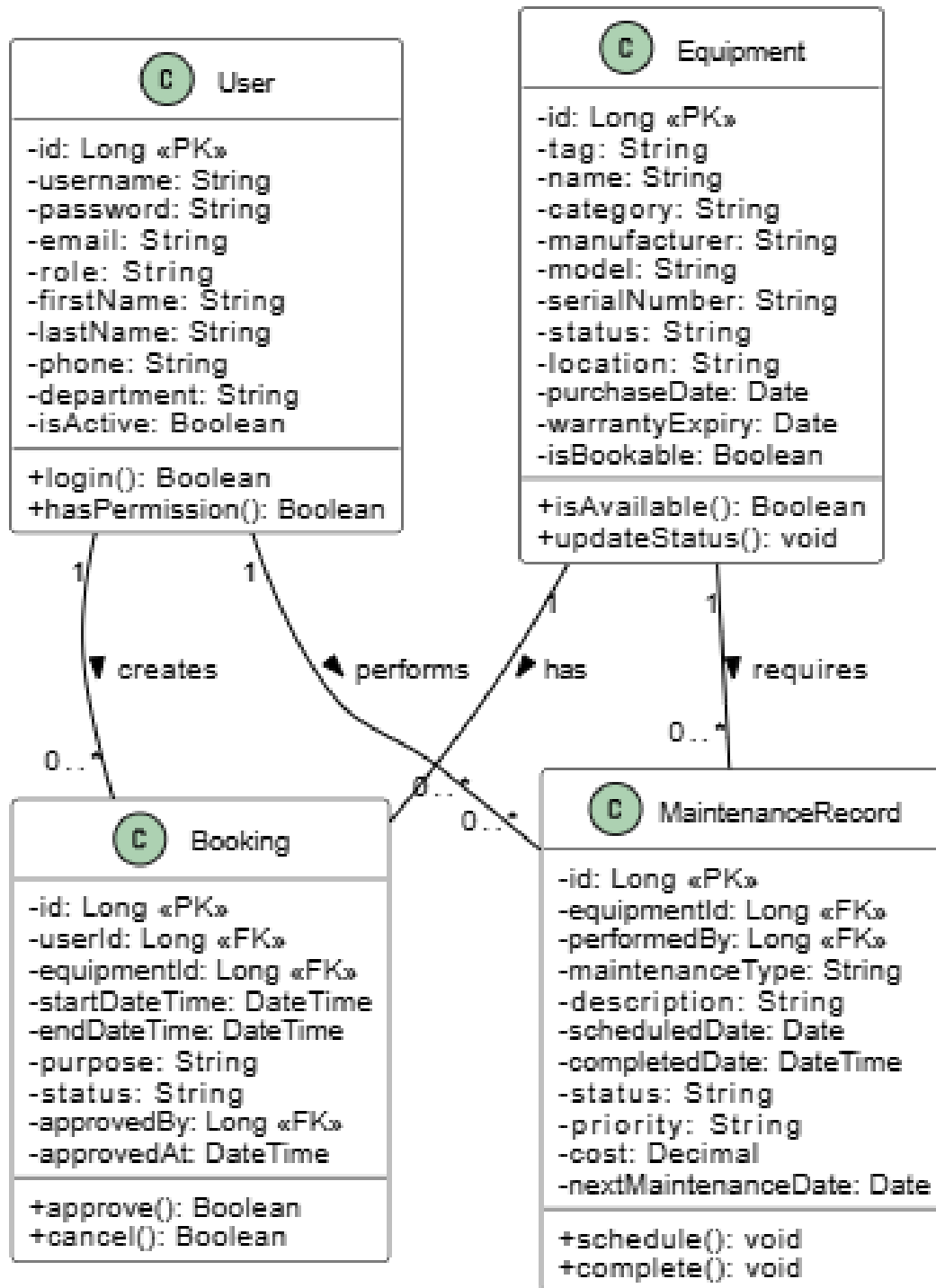
DESCRIPTION

LabMS consists of several integrated modules, each addressing a specific aspect of laboratory management:

- **Backend:** Built using Spring Boot (Java), the backend exposes RESTful APIs for user authentication, visitor management, equipment inventory, booking, and maintenance scheduling. It uses JPA repositories for data persistence and enforces business logic and validation.
- **Frontend:** The React-based frontend provides a modular, single-page application experience. It includes dedicated pages for dashboard, visitor management, equipment booking, and maintenance scheduling. Static HTML/JS pages are used for quick access to core features.
- **Database:** A relational database (MySQL or H2) stores all entities, including users, visitors, equipment, bookings, and maintenance records. Relationships are defined to ensure data integrity and support complex queries.
- **User Roles:** The system supports three main roles:
 - **Admin:** Full access to all modules, including user management, equipment inventory, and maintenance scheduling.
 - **Host:** Limited access to visitor management, allowing approval or rejection of visitor requests.
 - **Lab Technician:** Access to maintenance scheduling and equipment management.
- **Workflows:** The application supports end-to-end workflows, such as visitor registration and approval, equipment booking and conflict resolution, and maintenance scheduling with status updates (Scheduled, In Progress, Completed, Overdue).
- **Dashboards:** Real-time dashboards display key metrics, such as the number of visitors, equipment bookings, and maintenance tasks, with filters for date ranges and status.
- **Notifications:** The system provides notifications for pending approvals, upcoming maintenance, and booking conflicts, improving communication among users.

CHAPTER 4

CLASS DIAGRAM



CHAPTER 5

TABLE STRUCTURES

5.1 User

id	name	email	password	role	student_id	department	phone	is_active	is_email_verified
53	SUBASH MUTHU B	subashmuthu149@gmail.com	\$2b\$12\$STUIU7Y3vPmPee0vmIBSt.Mvm.pL1FG...	admin	NULL	NULL	NULL	1	1
54	admin	2312401@nec.edu.in	\$2b\$12\$pFa5KwOCH5F1ZKRXyz2eeOR/FkuExl...	admin	NULL	NULL	NULL	1	1
55	subikshan	2312501@nec.edu.in	\$2b\$12\$uJAZ1bZMdCmk8Thf/HJSuxl4RX7fgd0...	teacher	NULL	NULL	NULL	1	1
56	Alwin	2312404@nec.edu.in	\$2b\$12\$BxnQeXUUL8odmr15tK0a9eJry9nrCqOn...	student	NULL	NULL	NULL	1	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

5.2 Labs

id	name	description	location	lab_type	is_active	created_by	created_at	updated_at	capacity	square_feet	lab_seats
8	Cisco	Welcome	Ground floor	cse	1	35	2025-09-10 08:48:02	2025-10-22 15:21:44	100	455	67
9	APJ	greet	Ground floor ,Near dept office	cse	1	42	2025-10-14 15:25:36	2025-10-22 17:41:53	67	4520	25
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

5.3 Equipment details

id	name	description	serial_number	manufacturer	category	lab_id	location_details	status
9	computer001	NULL	44	DELL	computer	8	CSE	in_use
10	projector001	NULL	01	PowerPoint	projector	8	CSE	available
11	computer003	High-performance workstation	COMP-2024-003	DELL	computer	8	CSE Lab Station 3	in_use
12	computer004	Gaming workstation	COMP-2024-004	DELL	computer	8	CSE Lab Station 4	available
15	Dell OptiPlex 7090	High-performance desktop for CAD work	DELL-2024-001	Dell	computer	8	Room 101 Desk A1	available
16	HP EliteBook 840	Portable laptop for field research	HP-2024-002	HP	computer	8	Mobile Cart Station B	available
17	Epson PowerLite L3150	Ultra short throw classroom projector	EPSON-PRJ-001	Epson	projector	8	Lecture Hall Ceiling Mount	available
18	Canon ImageClass MF445dw	Multifunction laser printer with duplex	CANON-PRT-001	Canon	printer	8	Main Office Print Station	available
19	Nikon Edipse E200	Biological microscope for cell studies	NIKON-MIC-001	Nikon	microscope	8	Biology Lab Bench Station 1	available
20	Autoclave SX-500	Steam sterilization equipment	AUTOCLAVE-001	Tuttnauer	lab_equipment	8	Sterilization Room	available
22	xf	hdc	srff	srff	computer	9	hnd	available

5.4 Booking

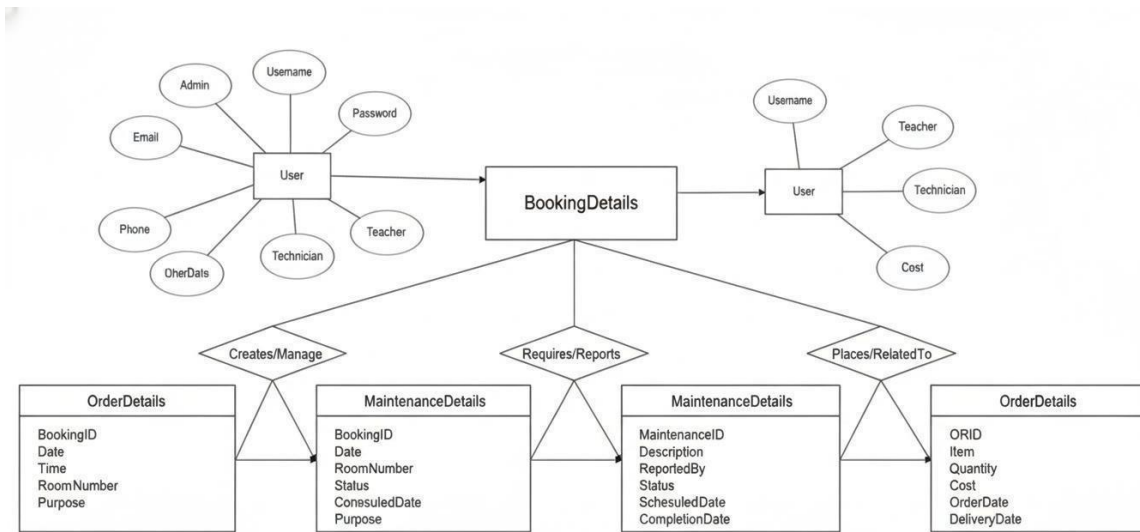
id	user_id	booking_type	lab_id	equipment_id	start_time	end_time	status	purpose	created_at	updated_at
20	54	lab	8	NULL	2025-12-13 04:30:00	2025-12-13 11:30:00	pending	model lab	2025-10-21 13:10:15	2025-10-21 13:10:15
21	54	equipment	9	121	2025-10-30 16:52:00	2025-10-30 17:30:00	cancelled	greet	2025-10-21 15:46:49	2025-10-22 14:45:59
22	54	lab	8	NULL	2025-12-04 01:34:00	2025-12-04 14:35:00	pending	sem lab	2025-10-23 01:35:13	2025-10-23 01:35:13
23	54	lab	8	NULL	2025-10-25 03:30:00	2025-10-25 12:30:00	pending	Drive	2025-10-23 19:39:16	2025-10-23 19:39:16
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

5.5 Notification

id	user_id	title	message	type	created_by	created_at	updated_at	read	r
12	54	Lab Booking Created	Your booking for Cisco has been created succes...	booking	NULL	2025-10-21 13:10:15	2025-10-21 13:10:15	0	0
13	54	Incident Reported	New medium priority incident "breakdown" has b...	incident	NULL	2025-10-21 15:04:53	2025-10-21 15:04:53	0	0
14	54	Maintenance Scheduled	Maintenance for Computer-001 has been sched...	maintenance	NULL	2025-10-21 15:06:04	2025-10-21 15:06:04	0	0
15	54	Equipment Booking Created	Your booking for HP EliteBook 840 has been cre...	booking	NULL	2025-10-21 15:46:49	2025-10-21 15:46:49	0	0
16	54	Lab Deleted	Lab "APJ" has been deleted.	lab	NULL	2025-10-22 13:21:10	2025-10-22 13:21:10	0	0
17	54	Lab Updated	Lab "Cisco" has been updated.	lab	NULL	2025-10-22 15:21:44	2025-10-22 15:21:44	0	0
18	54	Lab Updated	Lab "APJ" has been updated.	lab	NULL	2025-10-22 17:27:00	2025-10-22 17:27:00	0	0
19	54	Lab Updated	Lab "APJ" has been updated.	lab	NULL	2025-10-22 17:27:36	2025-10-22 17:27:36	0	0
20	54	Lab Updated	Lab "APJ" has been updated.	lab	NULL	2025-10-22 17:41:53	2025-10-22 17:41:53	0	0
21	54	Lab Booking Created	Your booking for Cisco has been created succes...	booking	NULL	2025-10-23 01:35:13	2025-10-23 20:01:14	1	0
22	54	Lab Booking Created	Your booking for Cisco has been created succes...	booking	NULL	2025-10-23 19:39:16	2025-10-23 19:39:16	0	0

CHAPTER 6

ER DIAGRAM



CHAPTER 7

TECH STACK

Frontend Technologies: React.js, Vite, Tailwind CSS

Backend Technologies: Node.js, Express.js

Middleware: Authentication Middleware, File Upload Middleware, CORS
Middleware, Body Parser

Database: MySQL

Packages Used: Sequelize ORM, bcrypt.js, jsonwebtoken, multer, dotenv, cors,
express-validator, nodemon

Approach: Follows Sequelize ORM with Async/Await programming, RESTful
API design, and Modular Architecture

Architecture: MVC (Model–View–Controller)

- **Models:** Sequelize models represent the database entities (User, Equipment, Booking, Maintenance, Notifications, etc.).
- **Views:** React components manage user interfaces and interactions.
- **Controllers:** Express controllers handle business logic, validation, and API responses.

CHAPTER 8

MODULES

8.1 HOME PAGE : (Landing Page)

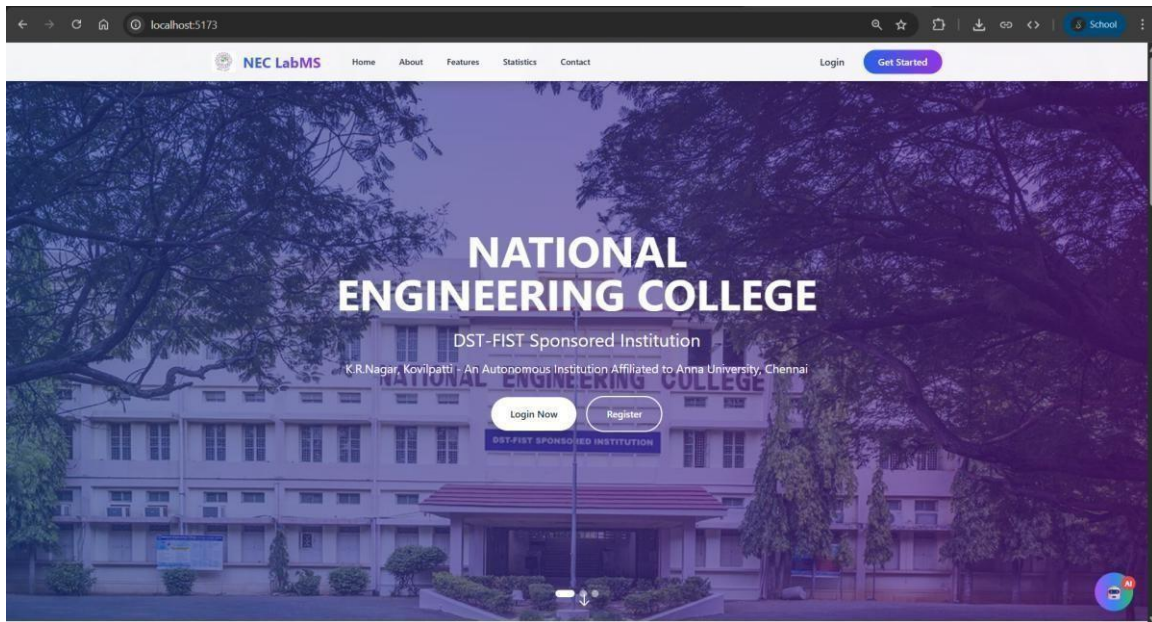
The Home Page serves as the public-facing entry point for the Laboratory Management System, providing visitors with system overview, key features, login/register access, and general information about the platform before authentication.

React Concepts

- **useState for Interactive Elements:** Manages `activeSection` for navigation highlighting, `showLoginModal` and `showRegisterModal` for authentication dialogs, `contactFormData` for inquiry submissions, and `isSubmitting` state during form processing.
- **Component Layout & Sections:** Implements Hero section with system title, tagline, and call-to-action buttons, Features section showcasing key modules with icons and descriptions, About section explaining system benefits, and Contact section with inquiry form.
- **Responsive Navigation:** Uses `useState` to manage `mobileMenuOpen` for hamburger menu toggle on small screens, smooth scroll implementation for section navigation links, sticky header that appears/hides on scroll.
- **Animations & Interactions:** Integrates animation libraries (`framer-motion`, `AOS`) for fade-in effects on scroll, hover effects on feature cards, parallax scrolling for hero section, and loading animations during page transitions.

Backend Integration

- **Fetch system information:** `GET /api/public/info` — Retrieves public system data including platform name, description, supported features list, contact information, and announcement messages without requiring authentication.
- **Submit contact inquiry:** `POST /api/public/contact` — Accepts contact form data with name, email, subject, message fields, validates email format and required fields, sends inquiry to admin email, stores in database for tracking, and returns success confirmation.
- **Check system status:** `GET /api/public/status` — Returns system availability status, scheduled maintenance notices, current version, and service health for display on public pages without authentication requirements.



(Fig 8.1 shows the home page)

8.2 LOGIN PAGE

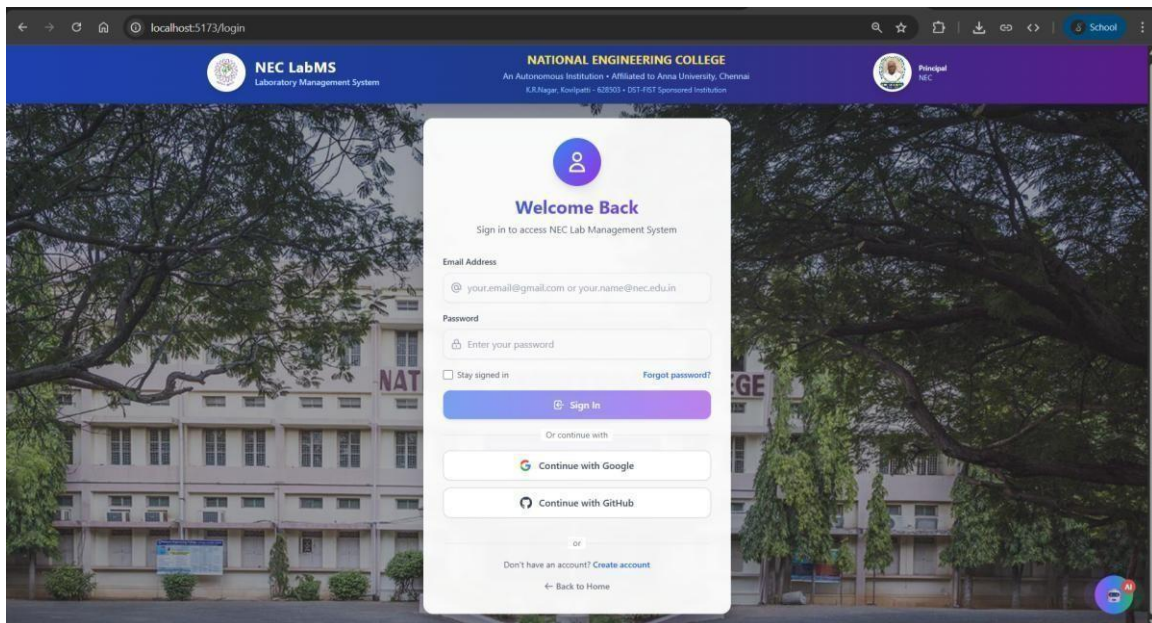
The Login Page provides secure user authentication interface where existing users enter credentials to access the Laboratory Management System with support for password recovery and remember me functionality.

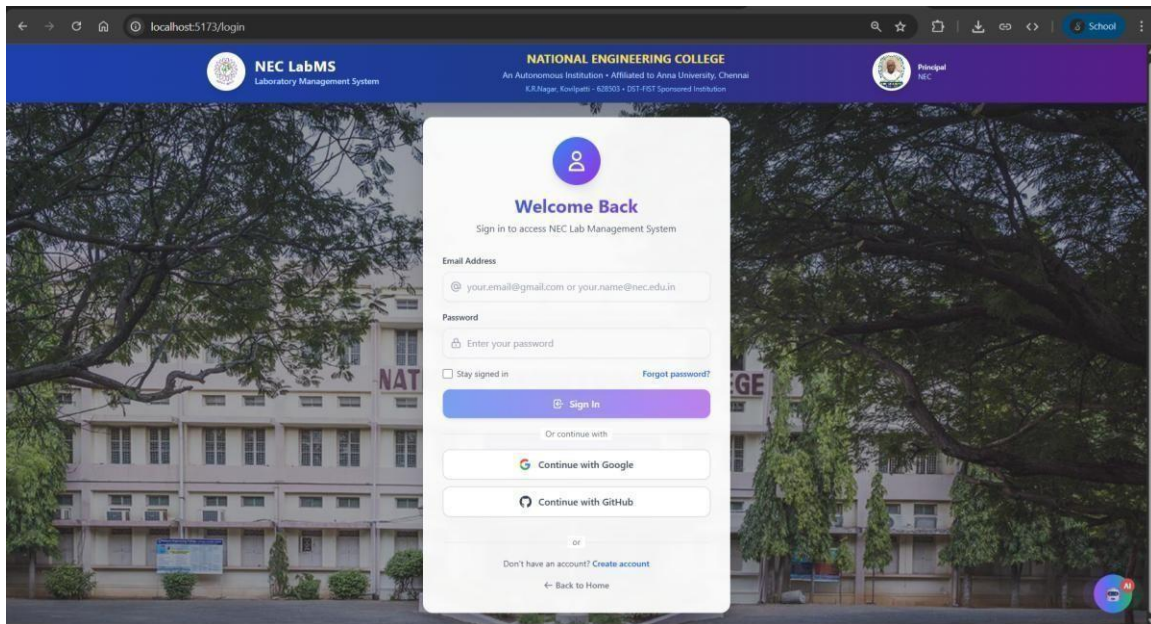
React Concepts

- **useState for Form Management:** Manages formData object with email and password fields, rememberMe boolean for persistent login, showPassword toggle for password visibility, validationErrors object for field-specific error messages, isLoading during authentication process, and loginError for failed attempts.
- **Controlled Form Inputs:** Implements email and password inputs with values bound to formData state, onChange handlers updating state on each keystroke, real-time validation showing errors below fields, submit button disabled when form invalid or submitting.
- **Password Visibility Toggle:** Eye icon button toggles showPassword state between true/false, changes password input type between "password" and "text", provides better UX for users checking typed password accuracy.
- **Error Handling & Feedback:** Displays authentication errors from backend (invalid credentials, account locked, account not verified) in prominent alert banner, clears errors when user modifies input, shows specific error messages guiding user to resolution.

Backend Integration

- User login: POST /api/auth/login — Accepts email and password in request body, validates credentials against database using secure password hashing (bcrypt), checks account status (active/locked/unverified), generates JWT authToken with user ID and role encoded, sets token expiration based on rememberMe option (24 hours vs 7 days), returns token and user profile data.
- Validate token: GET /api/auth/validate — Accepts authToken in Authorization header, verifies token signature and expiration, checks user still exists and active, returns user data if valid or 401 Unauthorized if invalid/expired for automatic logout.
- Request password reset: POST /api/auth/forgot-password — Accepts email address, validates user exists, generates secure reset token with expiration, sends password reset email with token link, returns success message without revealing if email exists for security.





(Fig 8.2 shows the login page)

8.3 REGISTER PAGE

The Register Page enables new users to create accounts in the Laboratory Management System by providing required information, selecting roles, agreeing to terms, and completing email verification for account activation.

React Concepts

- **useState for Registration Form:** Manages formData object with fields (name, email, password, confirmPassword, phone, department, position), showPassword and showConfirmPassword booleans, validationErrors object, agreedToTerms boolean, isRegistering loading state, and registrationSuccess flag.
- **Multi-Step Form Validation:** Implements real-time validation including email format checking with regex pattern, password strength validation (minimum 8 characters, uppercase, lowercase, number, special character), confirm password matching check, required field validation, phone number format validation.
- **Password Strength Indicator:** Creates visual password strength meter using strength calculation function, displays colored progress bar (red: weak, yellow: medium, green: strong), shows strength label text, and provides password requirements list with checkmarks.
- **Terms & Conditions:** Checkbox for terms acceptance with link to terms modal/page, disables submit button until checked, validates acceptance on form submission, and stores acceptance timestamp in registration data.

Backend Integration

- User registration: POST /api/auth/register — Accepts complete user data in request body, validates email uniqueness (returns error if already exists), validates required fields and formats, hashes password using bcrypt before storage, creates user account with "unverified" status, generates email verification token, sends verification email with activation link, returns success message instructing user to check email.
- Verify email: GET /api/auth/verify-email?token=abc123 — Accepts verification token from email link as query parameter, validates token exists and not expired (typically 24-48 hour expiration), updates user account status to "active", marks email as verified, returns success page or redirects to login with success message.
- Resend verification email: POST /api/auth/resend-verification — Accepts email address for unverified account, validates user exists and not yet verified, generates new verification token, sends new verification email, limits requests to prevent abuse (max 3 per hour), returns success confirmation
- Check email availability: GET /api/auth/check-email?email=[user@example.com](#) — Quick endpoint to check if email already registered, returns boolean availability status, used for real-time validation during registration form completion without full submission.

The screenshot shows a web browser window with the URL `localhost:5173/register`. The page features a registration form titled "Create Account" for a "Lab Management System". The form is set against a background image of a university building with trees. The form fields and controls are as follows:

- Full Name ***: A text input field with the placeholder "Enter your full name".
- Email Address ***: A text input field with the placeholder "Enter your email address (@gmail.co)". To its right is a "Send OTP" button.
- Password ***: A text input field with the placeholder "Create a password (min 6 characters)".
- Confirm Password ***: A text input field with the placeholder "Re-enter your password".
- Register As ***: A dropdown menu with "Student" selected.
- Buttons**: A large blue button labeled "Verify Email First" is positioned below the password fields. At the bottom of the form, there is a link "Already have an account? Sign in" and a link "Back to Home".

(Fig 8.3 shows the Register page)

8.4 DASHBOARD PAGE

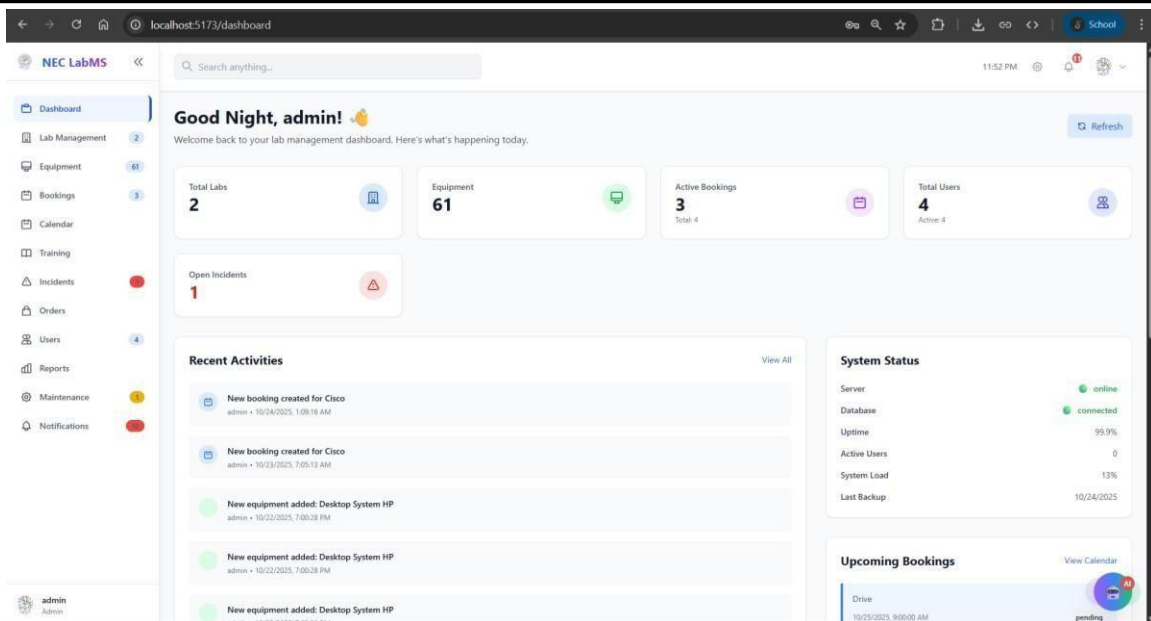
The Dashboard serves as the central command center providing real-time insights into laboratory operations, resource utilization, and system health through an intuitive interface with statistical cards and activity feeds.

React Concepts

- **useState & useEffect Hooks:** Manages dashboard statistics (total labs, equipment, bookings, users, incidents) and recent activity logs using `useState`. The `useEffect` hook fetches initial data on component mount and sets up 60-second polling intervals for automatic updates.
- **Component Composition:** Implements reusable `StatCard` components for displaying metrics with icons and trends, `ActivityListItem` components for showing recent events with timestamps, and `SystemStatusIndicator` components for server/database health monitoring with color-coded status badges.
- **Conditional Rendering:** Displays loading skeleton screens while data fetches from backend APIs, shows error messages with retry buttons when API calls fail, and renders empty state messages when no activities exist.

Backend Integration

- **Fetch dashboard metrics:** `GET /api/dashboard` — Retrieves comprehensive statistics including total laboratories, equipment count, active bookings, registered users, and open incidents. Response includes system health data like server uptime and database status.
- **Fetch recent activities:** `GET /api/activities?limit=10` — Returns the 10 most recent system activities across all modules including bookings, equipment changes, incidents, and maintenance tasks with user information and timestamps.
- **Authentication:** All dashboard requests require valid `authToken` in `Authorization` header for user authentication and access control.



(Fig 8.4 shows the Dashboard page)

8.5 LAB MANAGEMENT PAGE

The Lab Management module provides comprehensive control over laboratory spaces, allowing administrators to create, edit, delete, and organize lab records while enabling users to browse available laboratories for booking purposes.

React Concepts

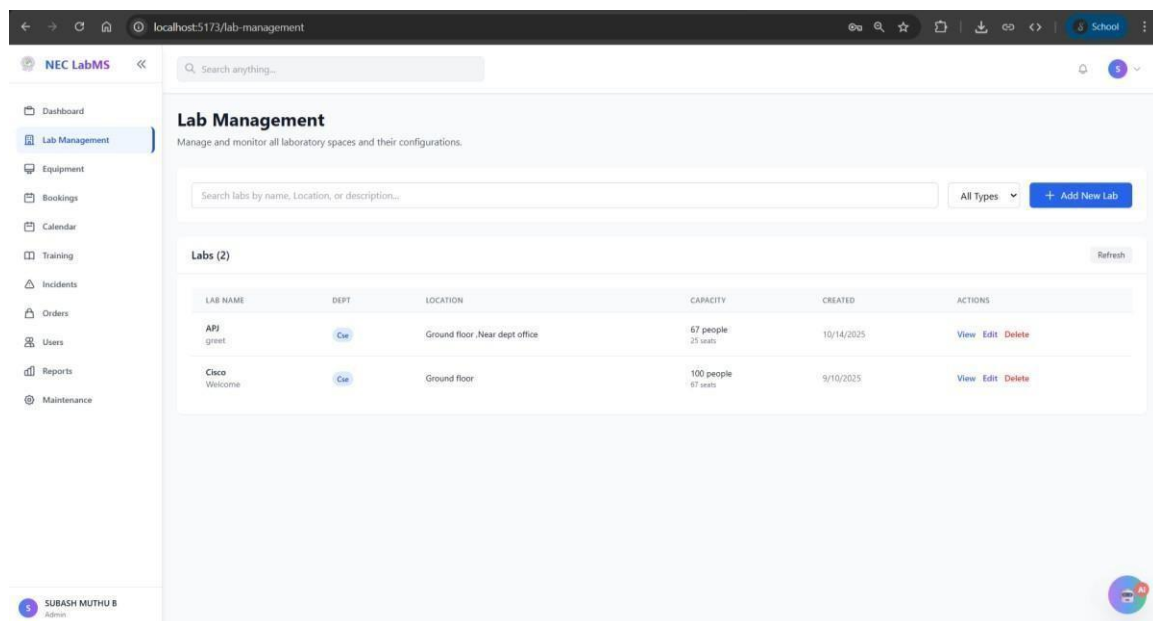
- **useState for State Management:** Maintains labs array with all laboratory records, filteredLabs after applying search/filters, searchQuery for text input, filters object for department/location/status criteria, selectedLab for editing, and showModal boolean for dialog visibility.
- **useEffect with Dependencies:** Fetches all laboratories on component mount, recalculates filtered results whenever labs array or filter criteria change, and implements debounced search to avoid excessive API calls during typing.
- **Controlled Forms & Modals:** Modal dialog contains form inputs with values bound to state and onChange handlers updating state. Form validation checks required fields (name, department, location, capacity) before submission.
- **Table with Sorting:** Implements sortable table headers tracking current sort column and direction using useState, with onClick handlers toggling sort direction and visual arrow indicators showing active column.

Backend Integration

- **Fetch all laboratories:** GET /api/labs?page=1&limit=20&sortBy=name&order=asc
— Retrieves paginated laboratory list with query parameters for pagination,

sorting, and filtering. Response includes labs array, total count, and page metadata.

- Create new laboratory: POST /api/labs — Accepts laboratory data with required fields (name, department, location, capacity) and optional fields (description, equipment types). Server validates unique names and returns created lab with generated ID.
- Update laboratory: PUT /api/labs/:id — Updates existing laboratory record with new field values. Backend validates authorization and returns updated lab object or error details.
- Delete laboratory: DELETE /api/labs/:id — Removes laboratory after checking for dependencies like active bookings. Returns 409 Conflict if dependencies exist, otherwise 204 No Content on success.
- Authentication & Authorization: All requests require authToken. Admin operations (create, update, delete) verify admin role, returning 403 Forbidden for unauthorized users.



(Fig 8.5 shows the Lab managing page)

8.6 EQUIPMENT INVENTORY PAGE

The Equipment Inventory module manages complete equipment lifecycle from acquisition to retirement, tracking location, status, maintenance history, and utilization for optimal resource allocation and operational readiness.

React Concepts

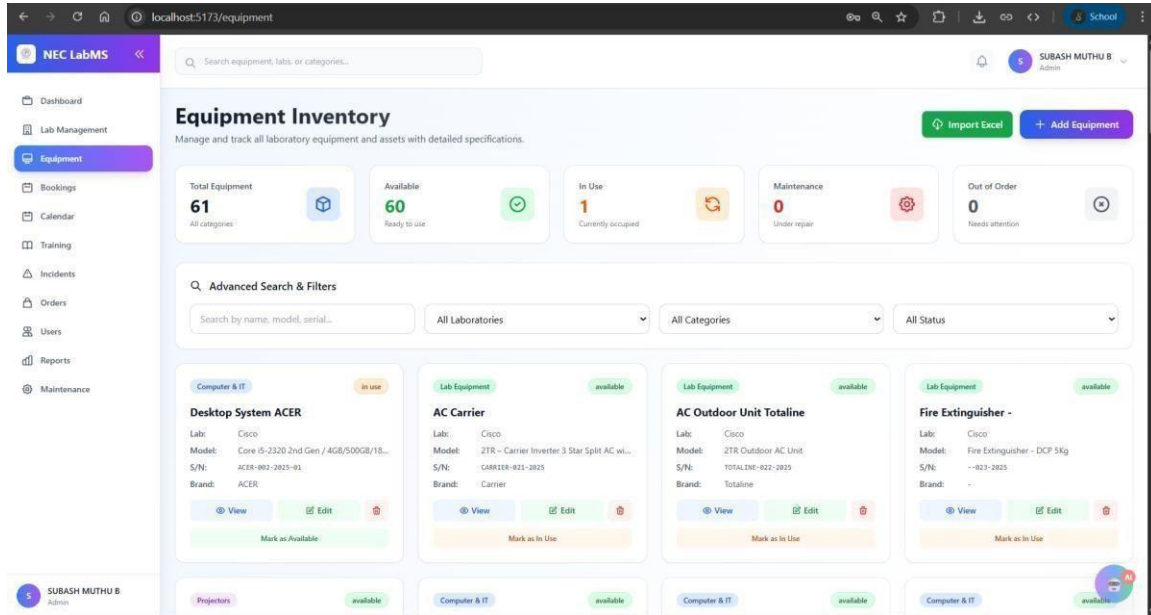
- **useState for Complex State:** Manages equipment array, filteredEquipment after search/filters, searchTerm, statusFilter (available, in-use, maintenance, out-of-order), categoryFilter, locationFilter, selectedEquipment for details, and viewMode (table/grid/list).
- **useEffect for Filtering:** Implements effect that recalculates filtered results when equipment array or any filter changes, applying search across name/model/serial and filtering by status/category/location.
- **File Upload Handling:** Import modal contains file input accepting .xlsx/.xls files, onChange captures file object, FileReader reads contents, and preview shows data before submission with progress indicators.
- **Dynamic View Rendering:** ViewMode state determines rendering between TableView for dense data with sortable columns, GridView for card-based layout with images, and ListView for simplified vertical layout.

Backend Integration

- **Fetch equipment inventory:** GET `/api/equipment?status=available&category=microscope&page=1` — Retrieves equipment with filtering by status, category, location, and pagination. Response includes equipment array with full details and metadata.
- **Create equipment:** POST `/api/equipment` — Accepts equipment data with required fields (name, category, serial_number, status, location) and optional fields (model, cost, warranty). Validates unique serial numbers and returns created record.
- **Update equipment status:** PATCH `/api/equipment/:id/status` — Updates equipment status, validates status transitions, creates associated maintenance/incident records, and sends notifications. Accepts new_status and optional notes.
- **Import equipment data:** POST `/api/equipment/import` — Accepts Excel file via multipart/form-data, parses and validates rows, performs bulk insert with

transaction management, and returns import summary with success count and errors.

- Export equipment data: GET /api/equipment/export?format=xlsx — Generates Excel file with filtered equipment data including formatted columns, status colors, and summary statistics with download headers.



(Fig 8.6 shows the equipment inventory page)

8.7 BOOKING PAGE

The Bookings module orchestrates laboratory and equipment reservations, implementing booking policies, approval workflows, conflict prevention, and utilization tracking through an intuitive reservation interface.

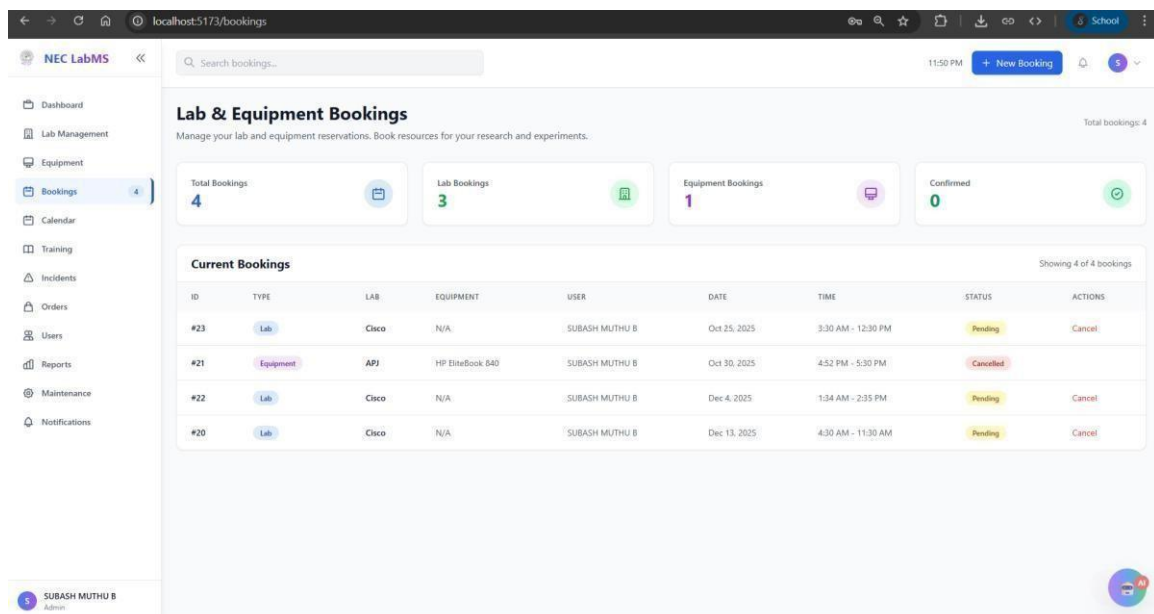
React Concepts

- useState for Booking Management: Maintains bookings array, userBookings for current user, filteredBookings after filters, showCreateModal, selectedBooking, formData with booking fields (resource_type, resource_id, start_datetime, end_datetime, purpose), availabilityData, and validationErrors object.
- Multi-Step Form Wizard: Manages currentStep state (1: select resource, 2: choose date/time, 3: add details, 4: review), implements Next/Previous navigation with step validation, and maintains formData across all steps.
- Real-Time Availability: Implements debounced effect triggering when resource or date/time changes, makes API call to check availability, updates availabilityData, and displays green/red indicators for available/unavailable slots.

Cancel for user's pending bookings, Approve/Reject for admins, Complete for active bookings, with disabled states and tooltips.

Backend Integration

- Fetch all bookings: GET `/api/bookings?status=pending&user_id=123&start_date=2025-01-01` — Retrieves bookings with filtering by status, user, resource, and date range. Response includes bookings array with user info, resource details, timestamps, and pagination metadata.
- Check availability: POST `/api/bookings/check-availability` — Accepts resource details and datetime in request body, queries existing bookings for conflicts, checks maintenance schedules and operating hours, returns `is_available` boolean and conflicting booking details if unavailable.
- Create new booking: POST `/api/bookings` — Validates resource availability, user permissions, booking duration limits, creates booking record with "pending" status, triggers approval workflow if required, and sends confirmation notifications.
- Cancel booking: DELETE `/api/bookings/:id` — Cancels booking after validating user ownership and cancellation policy compliance, updates resource availability, sends cancellation notifications, and returns success confirmation.



(Fig 8.7 shows the booking page)

8.8 CALENDER

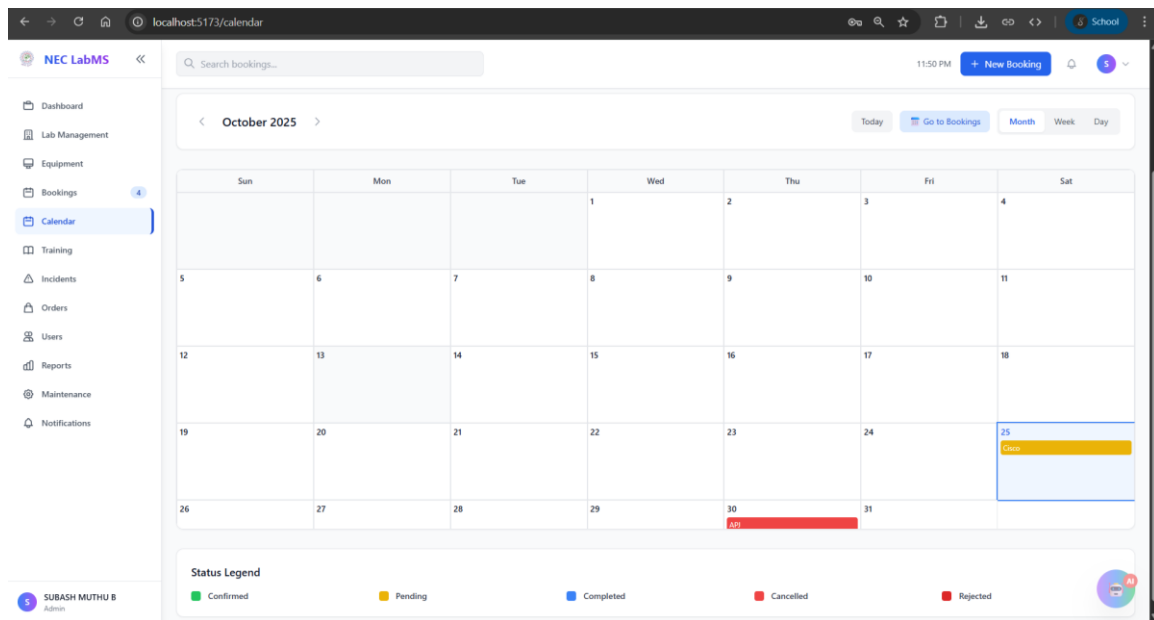
The Calendar module provides visual scheduling interface for understanding resource utilization patterns and availability across different time periods with interactive booking creation capabilities.

React Concepts

- `useState` for Calendar State: Manages `currentView` (day/week/month/agenda), `selectedDate` for active date, `calendarEvents` array with booking data, `selectedResource` for filtering, and `showBookingModal` when clicking time slots.
- Integration with Calendar Library: Uses third-party calendar component (`react-big-calendar` or similar) with custom event rendering, configures views and navigation, implements drag-and-drop for booking modification, and applies status-based color coding.
- Event Handling: Implements `onSelectSlot` handler for clicking empty time slots to create bookings, `onSelectEvent` for viewing booking details, `onNavigate` for date range changes triggering data refresh, and `onView` for switching between day/week/month views

Backend Integration

- Fetch calendar events: `GET /api/bookings?start_date=2025-01-01&end_date=2025-01-31&view=week` — Retrieves bookings within specified date range formatted for calendar display with event titles, start/end times, status colors, and resource information.
- Quick booking creation: `POST /api/bookings/quick` — Creates booking from calendar slot click with pre-filled date/time from selected slot, minimal required information, and immediate availability validation.
- Authentication: All calendar requests require `authToken` with `timezone` information for proper datetime handling across different user locations.



(Fig 8.8 shows the calendar page)

8.9 TRAINING PAGE

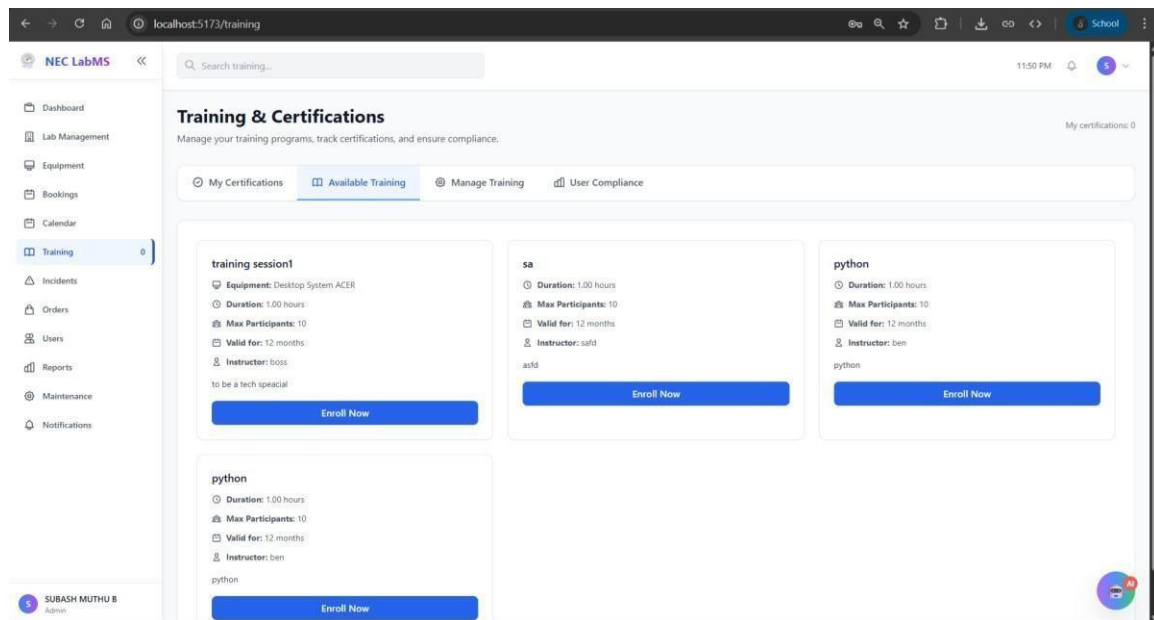
The Training & Certifications module manages laboratory safety training, equipment operation certifications, and compliance tracking to ensure users meet required qualifications before accessing equipment and facilities.

React Concepts

- **useState for Training Data:** Manages trainingPrograms array with available courses, userCertifications for logged-in user, enrollments for pending/active courses, filters (category, status, required/optional), selectedTraining for details, and showEnrollModal.
- **Training Program Display:** Renders training cards showing course name, description, duration, instructor, prerequisites, certification validity period, upcoming session dates, and enrollment status with available seats counter.
- **Enrollment Management:** Implements enrollment form with session selection dropdown, prerequisite checking that disables enrollment if requirements not met, waitlist option when sessions full, and confirmation dialog showing training details.
- **Progress Tracking:** Displays user's certifications with completion dates, expiration dates, renewal requirements, certification status badges (active, expiring soon, expired), and progress bars for in-progress training.

Backend Integration

- Fetch training programs: GET /api/training?category=safety&status=available — Retrieves training courses with filtering by category (safety, equipment, compliance), status (available, full, completed), and required/optional designation. Response includes program details, session schedules, and enrollment availability.
- Enroll in training: POST /api/training/:id/enroll — Validates prerequisites met, checks session availability, creates enrollment record, sends confirmation email with session details and preparation requirements, and returns enrollment confirmation.
- Fetch user certifications: GET /api/training/my-certifications — Returns current user's training history including completed courses, active certifications with expiration dates, in-progress enrollments, and required renewals with filtered views.
- Authentication: All training requests require authToken. Admins can create/manage training programs and view all user certifications, regular users access their own training records and available programs.



(Fig 8.9 shows the training page)

8.10 INCIDENT PAGE

The Incident Management module provides structured reporting, tracking, and resolution of laboratory incidents from equipment malfunctions and safety concerns to accidents and hazardous situations.

React Concepts

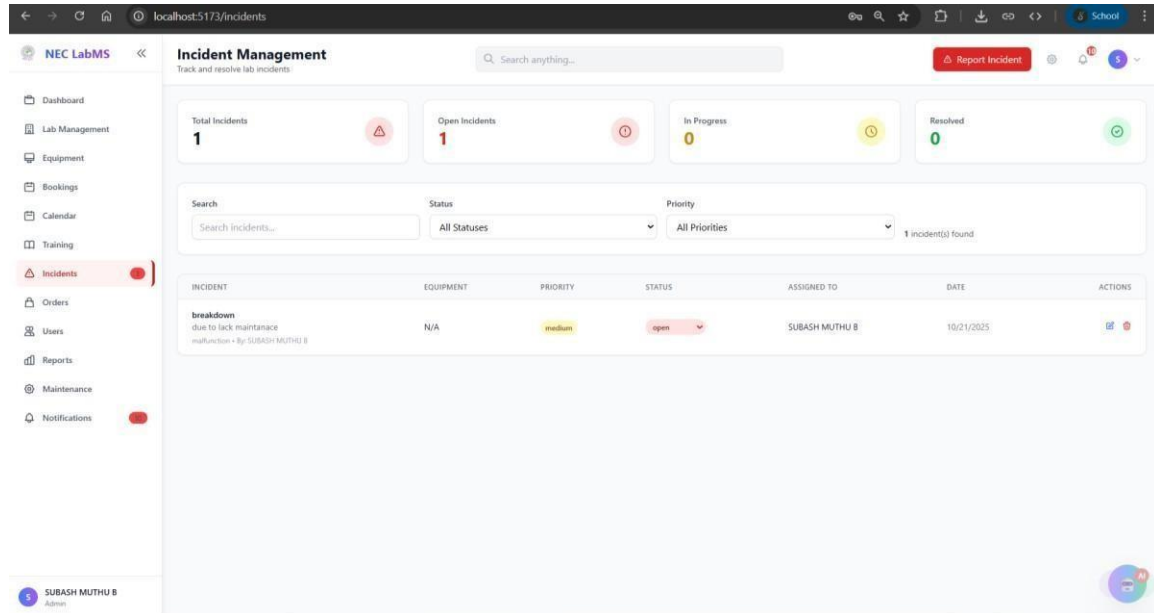
- **useState for Incident Tracking:** Manages incidents array, filteredIncidents, filters (type, severity, status, location, date range), selectedIncident for details, showReportModal, and formData with incident fields (type, severity, location, description, affected_equipment, immediate_actions).
- **Controlled Form for Reporting:** Implements incident type selection from hierarchical categories, severity assessment (minor, moderate, serious, critical), affected location/equipment selection, description textarea with character count, file upload for photos/documentation, and validation ensuring required fields.
- **Priority Classification:** Displays incidents with severity badges using color coding (green for minor, yellow for moderate, orange for serious, red for critical), filters by priority for urgent issue identification, and shows assignment status.
- **Status Lifecycle Display:** Renders incidents with status progression (reported, under investigation, in progress, resolved, closed, escalated), shows status history timeline, and enables status updates with required documentation.

Backend Integration

- **Fetch all incidents:** GET `/api/incidents?severity=critical&status=open&location=Lab-A` — Retrieves incidents with filtering by type, severity, status, location, date range, and assigned responder. Response includes incident details, reporter info, assignment data, and resolution documentation.
- **Report new incident:** POST `/api/incidents` — Creates incident record with classification validation, generates unique incident identifier, creates audit log entry, triggers notification workflow based on severity (escalation for critical incidents), and returns created incident with assigned responder.
- **Update incident status:** PATCH `/api/incidents/:id/status` — Updates incident status with workflow validation, requires investigation findings for resolved status, maintains comprehensive audit trail, and sends status change notifications

to stakeholders.

- **Authentication & Authorization:** All requests require authToken. Users can report incidents and view their own, admins and safety officers can view all incidents and update statuses, with role-based access control.



(Fig 8.10 shows the incident page)

8.11 ORDER MANAGEMEN

The Orders module manages procurement and acquisition of laboratory equipment, supplies, and services through complete lifecycle tracking from requisition to delivery and payment processing.

React Concepts

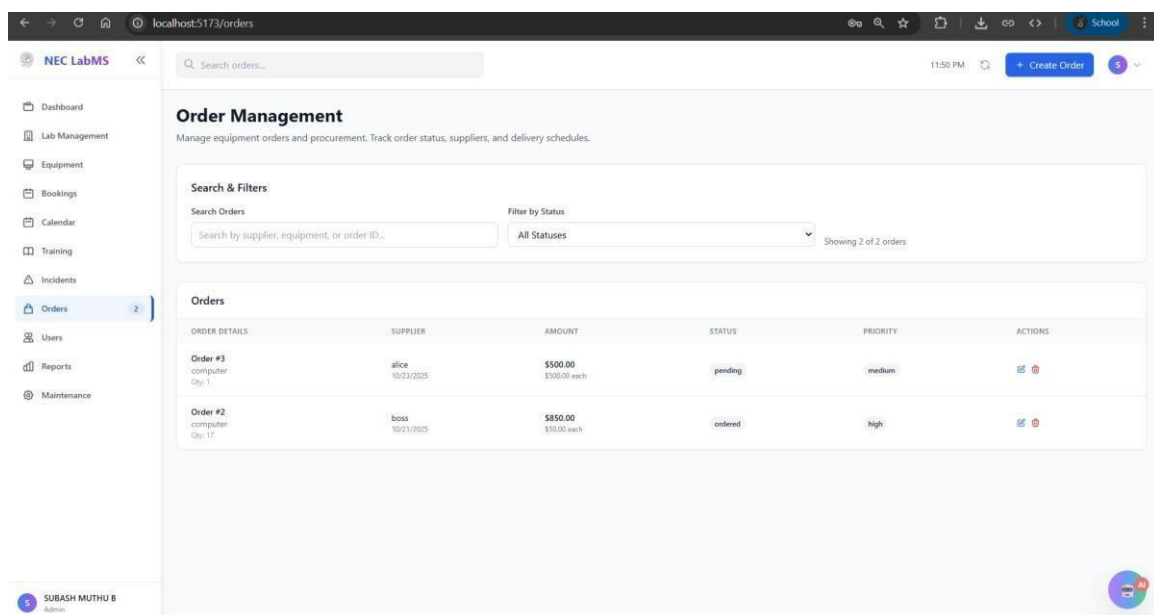
- **useState for Order Management:** Manages orders array, filteredOrders, filters (status, supplier, date range, amount range), selectedOrder for details, showCreateModal, and formData with order fields (items array, quantities, supplier_id, amount, priority, budget_account, delivery_date).
- **Multi-Item Order Form:** Implements dynamic item list where users can add/remove items, each with description/quantity/price, calculates total amount automatically when item values change, validates quantities as positive numbers, and shows running total.
- **Status Workflow Display:** Shows orders with status badges (requisition, pending approval, approved, ordered, partially received, received, completed, cancelled), implements approval routing for orders above thresholds, and displays approval

chain with timestamps.

- Supplier Selection: Dropdown shows approved suppliers with company names and contact info, displays supplier ratings and historical performance, filters suppliers by category, and shows pricing agreements if available.

Backend Integration

- Fetch all orders: GET `/api/orders?status=pending&supplier_id=456&start_date=2025-01-01` — Retrieves orders with filtering by status, supplier, date range, amount range, and requesting department. Response includes order details, item lists, supplier information, approval status, and delivery tracking.
- Create new order: POST `/api/orders` — Validates budget availability and purchasing authority limits, checks supplier approval status, creates order record with line items, initiates approval workflow if amount exceeds threshold, and returns order with generated purchase order number.
- Update order status: PATCH `/api/orders/:id/status` — Updates order through lifecycle stages, validates status transitions, records receipt information for delivered items, triggers payment processing for completed orders, and maintains audit trail with user and timestamp.
- Authentication & Authorization: All requests require authToken. Regular users can create requisitions, department heads approve within limits, purchasing staff manage supplier orders, with role-based authorization checks.



(Fig 8.11 shows the order management page)

8.12 REPORT AND ANALYTICS PAGE

The Reports & Analytics module generates comprehensive data analysis and exports for laboratories, equipment, bookings, maintenance, orders, and incidents supporting decision-making and compliance reporting.

React Concepts

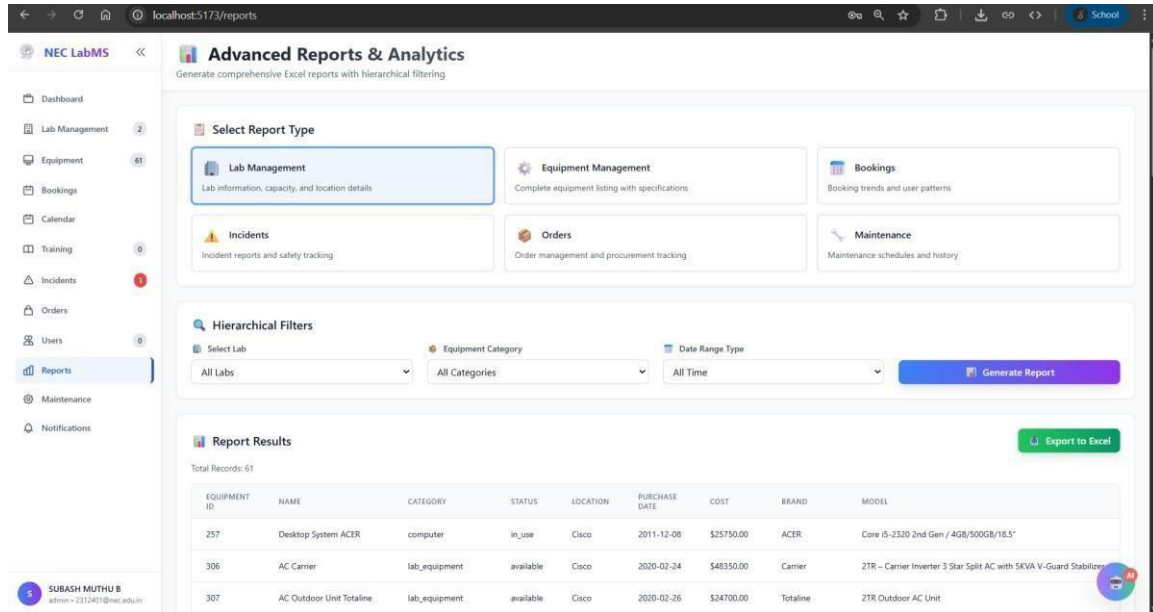
- **useState for Report Configuration:** Manages reportType selection (equipment utilization, booking statistics, maintenance summary, incident analysis, financial), filters object with date ranges/categories/locations, reportData with generated results, chartConfig for visualization settings, and isGenerating loading state.
- **Dynamic Filter Form:** Implements filter form that changes based on selected report type, shows relevant date range pickers, dropdown filters for labs/equipment/users, category selections, and generates button that validates required filters.
- **Data Visualization:** Integrates charting library (Recharts, Chart.js) for rendering bar charts showing usage trends, line charts for time-series analysis, pie charts for distribution, and data tables with sorting and pagination.
- **Export Functionality:** Implements export buttons for different formats (Excel, PDF, CSV), triggers report generation with applied filters, downloads generated files with proper naming (Equipment_Report_2025-01-15.xlsx), and shows download progress.

Backend Integration

- **Generate report:** POST /api/reports/generate — Accepts report type and filter parameters in request body, performs data aggregation queries across relevant tables, calculates statistics and trends, formats results according to report type, and returns JSON data for display.
- **Export report:** GET /api/reports/export?type=equipment&format=xlsx&start_date=2025-01-01&end_date=2025-01-31 — Generates downloadable report file with applied filters, creates formatted Excel workbook with multiple sheets (summary, details, charts), sets proper content-type and content-disposition headers, and streams file for download.
- **Fetch report templates:** GET /api/reports/templates — Returns available report

Types with descriptions, required filters, available formats, and sample output previews to help users select appropriate reports.

- **Authentication & Authorization:** All report requests require authToken. Financial and sensitive reports require admin role, standard operational reports available to lab managers, basic usage reports accessible to all authenticated users.



(Fig 8.12 shows the report & analysis page)

8.13 MAINTENANCE PAGE

The Maintenance Schedule module manages preventive and corrective maintenance for equipment, ensuring optimal operation, minimizing failures, and maintaining regulatory compliance with service requirements.

React Concepts

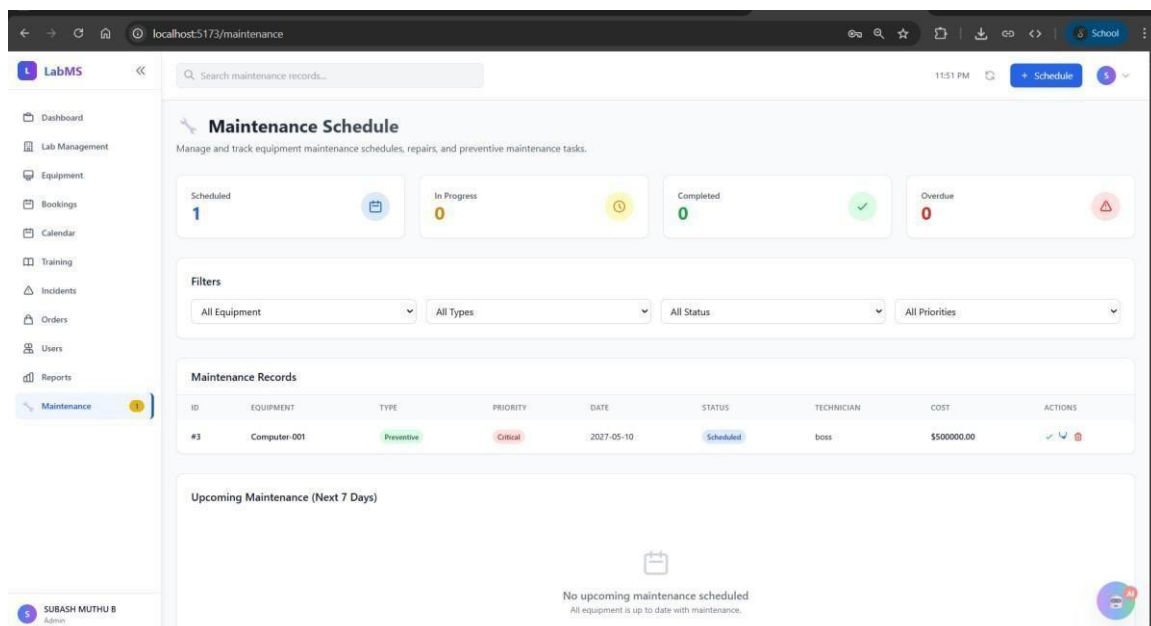
- **useState for Maintenance Data:** Manages maintenanceRecords array, filters object (equipment, type, status, priority, date range), selectedMaintenance for editing, showScheduleModal, and formData with task details (equipment_id, type, scheduled_date, technician_id, priority).
- **useEffect for Data Management:** Fetches maintenance records on mount, recalculates filtered results when filters change, identifies overdue tasks by comparing scheduled dates with current date, and polls for status updates.
- **Form with Date Selection:** Implements date picker for scheduling, technician

dropdown with availability checking, maintenance type selection (preventive, corrective, calibration, inspection),

- **Status Badges & Filtering:** Displays color-coded status badges (scheduled, due soon, overdue, in progress, completed, cancelled), implements filter tabs for quick status filtering, and shows counts for each status category.

Backend Integration

- **Fetch maintenance records:** GET `/api/maintenance?status=overdue&equipment_id=123&start_date=2025-01-01` — Retrieves maintenance tasks with filtering by status, equipment, technician, priority, and date range. Response includes task details, equipment info, technician assignments, and completion documentation.
- **Schedule maintenance:** POST `/api/maintenance` — Creates new maintenance task with equipment validation, technician availability checking, schedule conflict detection, recurring schedule setup if specified, and automatic notification to assigned technician.
- **Update maintenance status:** PATCH `/api/maintenance/:id/status` — Updates task status from scheduled to in-progress to completed, requires completion documentation for finished tasks, updates equipment status if needed, and maintains audit trail.
- **Fetch upcoming maintenance:** GET `/api/maintenance/upcoming?days=7` — Returns maintenance tasks due within specified timeframe for dashboard widgets and notification systems with prioritized sorting.



(Fig 8.13 shows the maintenance page)

8.14 NOTIFICATION PAGE

The Notifications module provides real-time alerts and messaging for bookings, maintenance, incidents, system updates, and important announcements to keep users informed of relevant events and required actions.

React Concepts

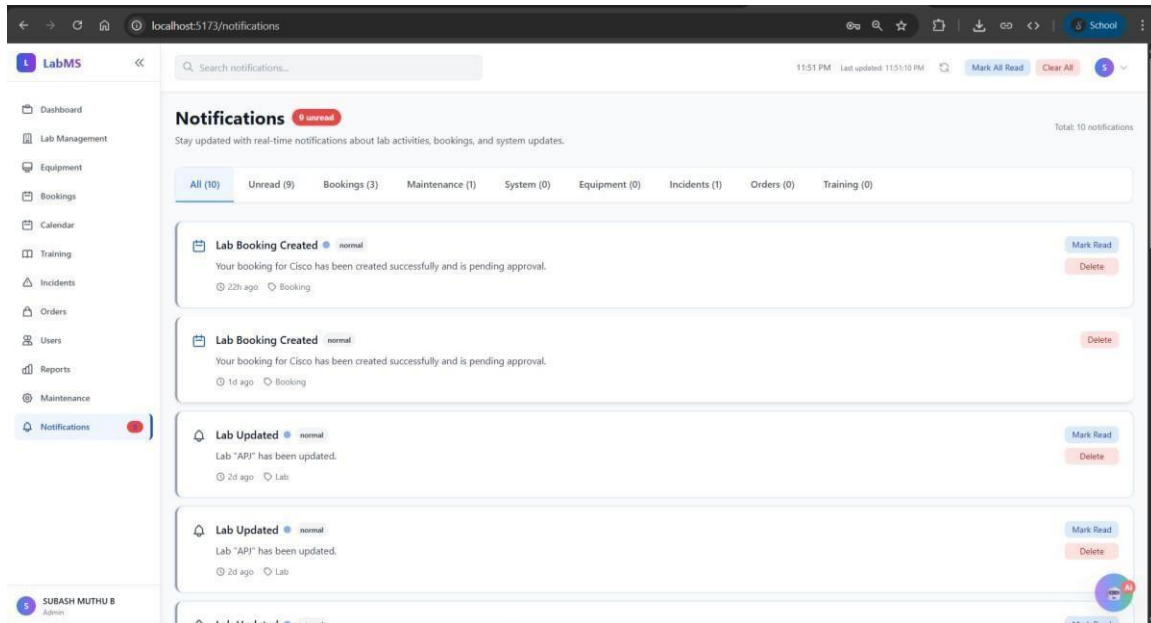
- **useState for Notification Management:** Manages notifications array with all user notifications, unreadCount for badge display, filters (type, status), selectedFilter for active tab (all, unread, bookings, maintenance, incidents), showNotificationPanel boolean for dropdown visibility.
- **Real-Time Updates:** Implements WebSocket connection or polling mechanism for push notifications, plays notification sound for high-priority alerts, displays browser notifications using Notification API with user permission, and shows toast messages for immediate alerts.
- **Notification List Rendering:** Displays notifications with type icons, message text, timestamp with relative time (5 minutes ago, 2 hours ago), unread indicator dot, and action buttons (mark read, delete, view details).
- **Filtering & Actions:** Implements filter tabs for notification types, mark all as read functionality, individual notification actions, bulk delete for selected notifications, and pagination for large notification lists.

Backend Integration

- **Fetch notifications:** GET /api/notifications?type=booking&status=unread&limit=20 — Retrieves user's notifications with filtering by type (booking, maintenance, incident, system, announcement), status (read/unread), and pagination. Response includes notification details, timestamps, related resource links, and unread count.
- **Mark as read:** PATCH /api/notifications/:id/read — Updates notification read status to true, returns updated notification object, updates unread count, and syncs across user's devices if multiple sessions active.

Delete notification: DELETE /api/notifications/:id — Removes notification from ser's list, returns success confirmation, updates unread count if notification was unread, and maintains notification in system logs for audit purposes.
- **WebSocket connection:** WS /api/notifications/stream — Establishes real-time connection for push notifications, authenticates with authToken, sends new

notifications immediately when events occur, maintains connection with heartbeat messages, and reconnects automatically on disconnection.



(Fig 8.14 shows the Notification page)

8.15 PROFILE PAGE

The User Profile & Management module allows users to view and edit personal information, upload profile pictures, and manage account settings while providing administrators with user account management capabilities.

React Concept

- **useState for Profile Data:** Manages profileData object with user fields (name, email, phone, department, position, bio), originalData for change detection, isEditing boolean, hasChanges tracking unsaved modifications, uploadedAvatar for image preview, and isSaving loading state.
- **Controlled Form with Auto-Save:** Implements input fields with values bound to profileData, onChange handlers updating state and setting hasChanges true, auto-save effect that triggers 2 seconds after user stops typing, saves changes automatically without submit button.
- **Avatar Upload:** File input accepts image types (jpeg, png), onChange reads file with FileReader, shows preview using createObjectURL, validates file size (max 5MB), compresses if needed, and uploads via separate API endpoint.
- **Admin User Management:** Admin view shows searchable user table, allows creating/editing/deactivating users, assigns roles (admin, lab manager, technician, user), manages permissions, and views user activity logs.

Backend Integration

- Fetch user profile: GET /api/users/profile — Retrieves current user's profile data including personal information, department/position, account settings, assigned roles/permissions, and profile picture URL with authentication via authToken.
- Update profile: PATCH /api/users/profile — Updates user profile with changed fields only, validates email format and uniqueness, phone number format, required fields, returns updated profile object, and maintains update audit trail.
- Upload avatar: POST /api/users/profile/avatar — Accepts image file via multipart/form-data, validates file type and size, processes and resizes image to standard dimensions, stores in cloud storage or file system, updates user record with new avatar URL, and returns new avatar URL.
- Admin fetch all users: GET
/api/users?role=user&department=science&status=active — Retrieves all users with filtering by role, department, status (active/inactive), and search query. Response includes user list with roles, last login, created date for admin management.
- Admin update user: PUT /api/users/:id — Allows admins to update any user's information, assign/remove roles, activate/deactivate accounts, reset passwords, requires admin authorization, and logs all admin actions for security audit.

The screenshot shows a web browser window with the address bar displaying 'localhost:5173/profile'. The page title is 'My Profile'. The main content area has a light blue background. A white card contains the profile information for 'admin'. At the top of the card is a circular profile picture placeholder with a camera icon and instructions: 'Click camera icon or drag & drop to upload', 'Images are saved automatically', 'Auto-compressed to 300x300px for optimal performance', and 'Max size: 10MB • Formats: PNG, JPG, GIF, WEBP'. Below this are input fields for 'Full Name (auto-saves)' with the value 'admin', 'Email (auto-saves)' with the value '2312401@nec.edu.in', 'Phone (auto-saves)' with the placeholder 'Enter your phone number', 'Department (auto-saves)' with the placeholder 'Enter your department', 'Position (auto-saves)' with the placeholder 'Enter your position/title', and a 'Bio (auto-saves)' text area with the placeholder 'Tell us about yourself...'. At the bottom left of the card, it says 'Real-time auto-save enabled' and 'Changes are automatically saved 2 seconds after you stop typing'. At the bottom right is a blue 'Save Now' button. A small circular icon is visible in the bottom right corner of the page.

(Fig 8.15 shows the Profile page)

8.16 SETTING PAGE

The System Settings module provides administrators with centralized configuration management for operating hours, booking policies, notification preferences, maintenance schedules, security settings, and holiday calendars.

React Concepts

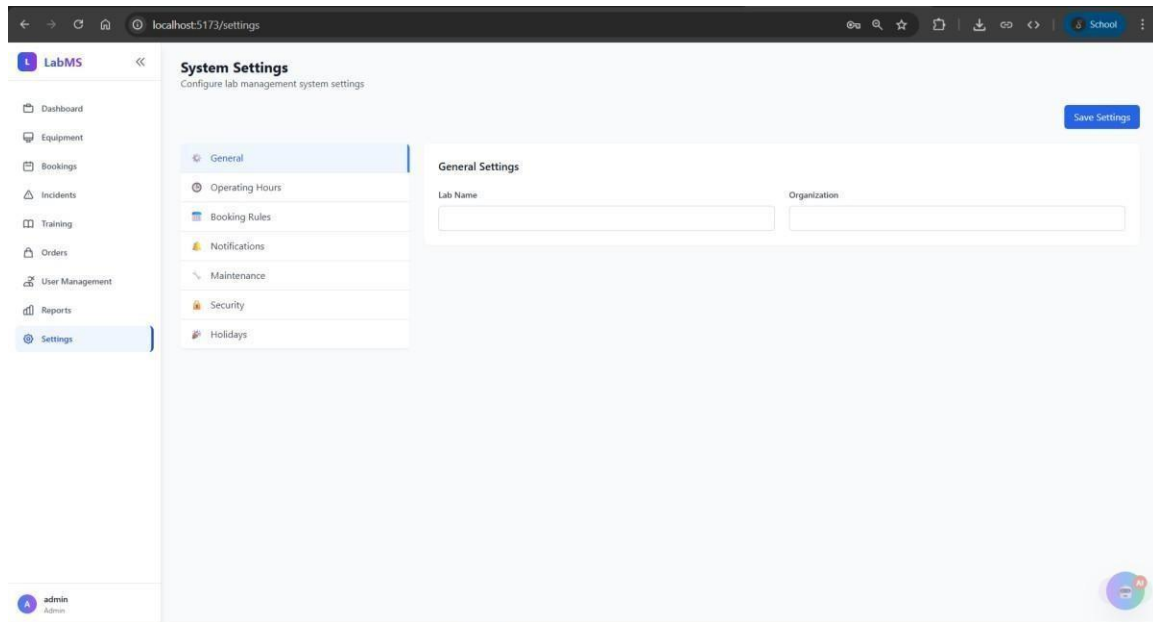
- **useState for Settings Management:** Manages settings object with all configuration sections (general, operating_hours, booking_rules, notifications, maintenance, security, holidays), activeTab for current settings category, hasUnsavedChanges tracking modifications, validationErrors for field-level errors, and isSaving state.
- **Tabbed Interface:** Implements tab navigation showing different settings categories, switches between setting sections without page reload, maintains state across tabs, highlights active tab, and shows unsaved changes indicator on tabs with modifications.
- **Dynamic Form Inputs:** Renders different input types based on setting configuration including text inputs for simple values, number inputs for numeric settings, toggles for boolean options, time pickers for operating hours, multi-select for notification channels, and date pickers for holidays.
- **Validation & Save:** Implements field-level validation ensuring valid time ranges (opening before closing), reasonable numeric limits (min < max booking duration), required field checking, shows inline error messages, validates entire form before save, and confirms before applying changes.

Backend Integration

- Fetch all settings: GET /api/settings — Retrieves complete system configuration including general settings (system name, timezone, language), operating hours per day of week, booking rules (max duration, advance booking window, cancellation policy), notification preferences, maintenance defaults, security settings (session timeout, password rules), and holiday list.
- **Update settings:** PUT /api/settings — Accepts updated settings object, validates all values against constraints, applies changes atomically (all or nothing), triggers system reconfiguration if needed, logs setting changes with admin user and timestamp, and returns updated settings confirmation.
 - **Fetch holidays:** GET /api/settings/holidays?year=2025 — Retrieves holiday calendar for specified year with holiday dates, names, and types

(public, institutional) affecting lab availability and booking rules.

- **Add/Remove holidays:** POST /api/settings/holidays & DELETE /api/settings/holidays/:id — Manages holiday calendar entries, validates date formats, prevents duplicate holidays, updates booking system to mark holidays unavailable, and returns updated holiday list.
- **Authentication & Authorization:** All settings endpoints require authToken with admin role verification. Non-admin users attempting to access return 403 Forbidden. All setting changes are logged with admin identification for accountability and audit compliance.



(Fig 8.16 shows the System Settings page)

CHAPTER 10

CONCLUSION

Lab_MS successfully integrates visitor management, equipment booking, and maintenance scheduling into a unified platform for laboratory operations. The system improves efficiency, transparency, and security by digitizing manual processes and providing real-time dashboards and reporting. Role-based access ensures that users interact with the system according to their responsibilities, while responsive design and robust error handling enhance user experience. The project lays a strong foundation for future enhancements, such as advanced analytics, automated notifications, and integration with institutional systems.

